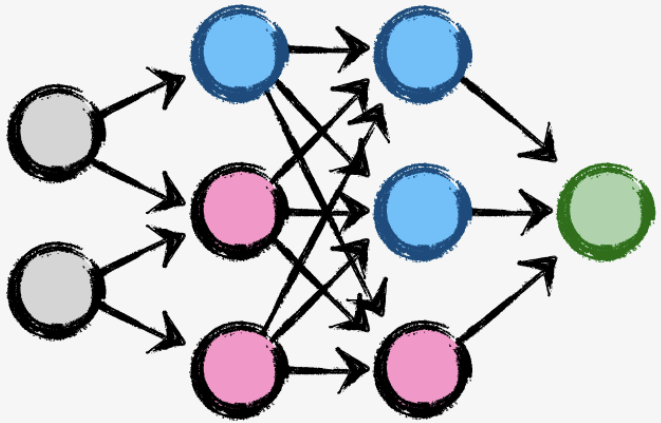
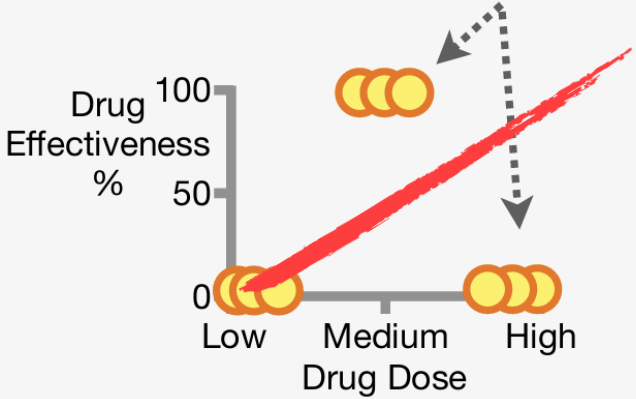
07

Neural Networks

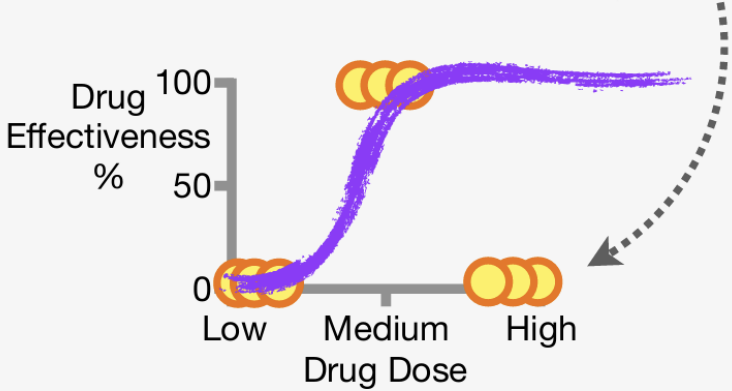


Neural Networks: Main Ideas

1 **The Problem:** We have data that show the Effectiveness of a drug for different Doses….



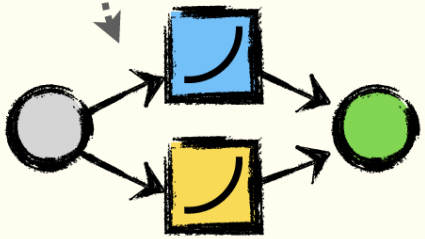
…and the **s-shaped squiggle** that **Logistic Regression** uses would not make a good fit. In this example, it would misclassify the high Doses.



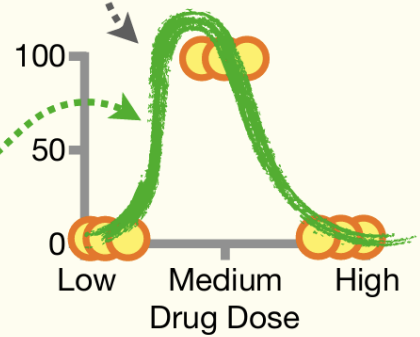
Neural Networks: Main Ideas

A Solution: Although they sound super intimidating, all **Neural Networks** do is fit **fancy squiggles** or **bent shapes** to data.

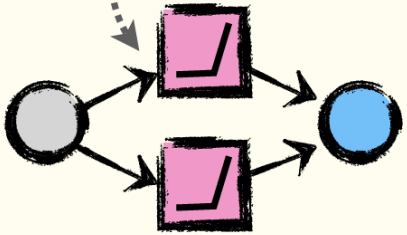
For example, we could get this **Neural Network**…



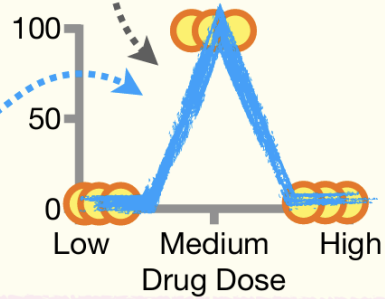
…to fit a **fancy squiggle** to the data…



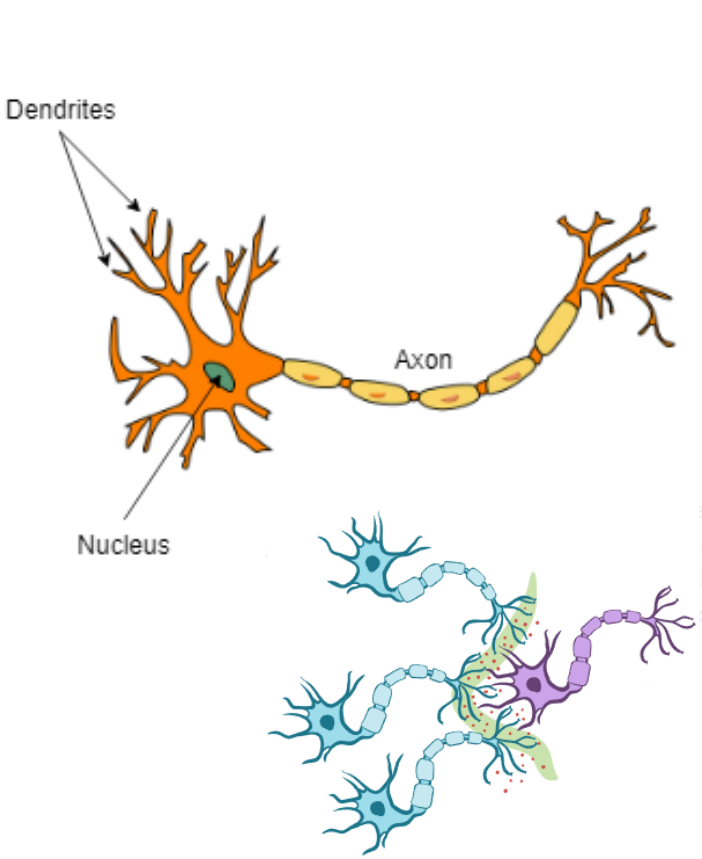
…or we could get this **Neural Network**…

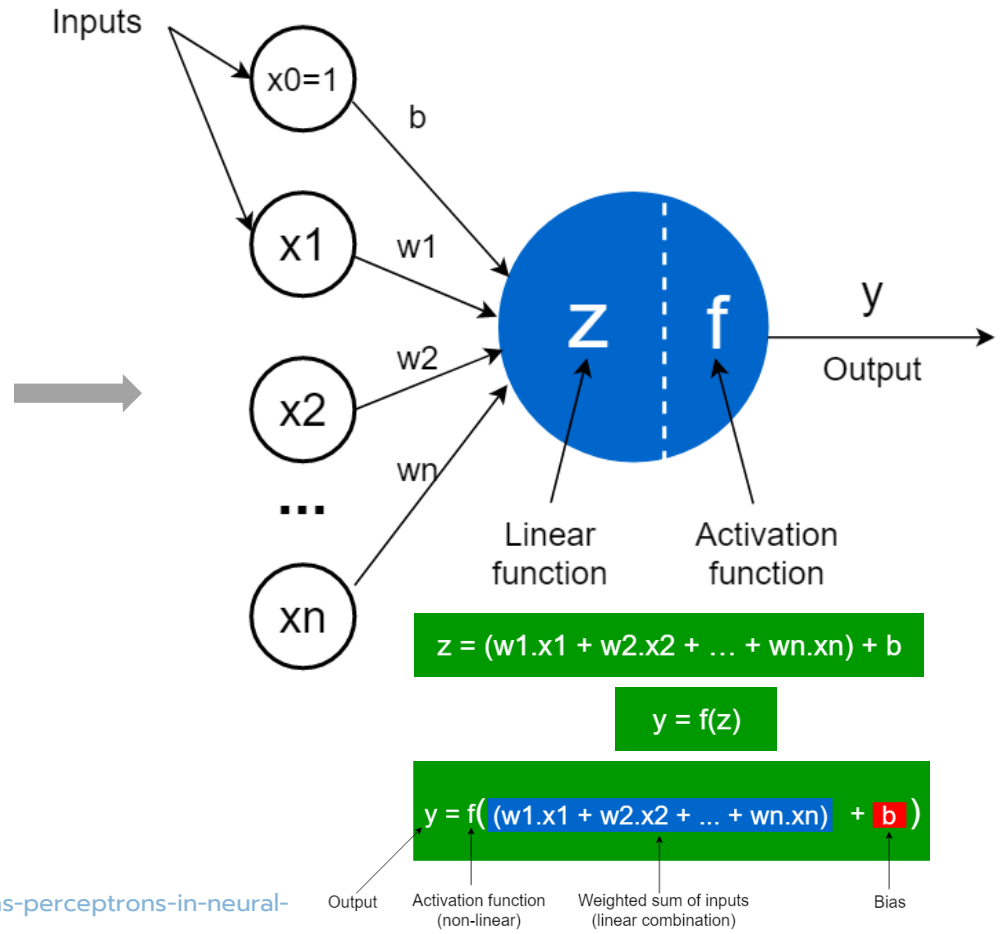


…to fit a **bent shape** to the data



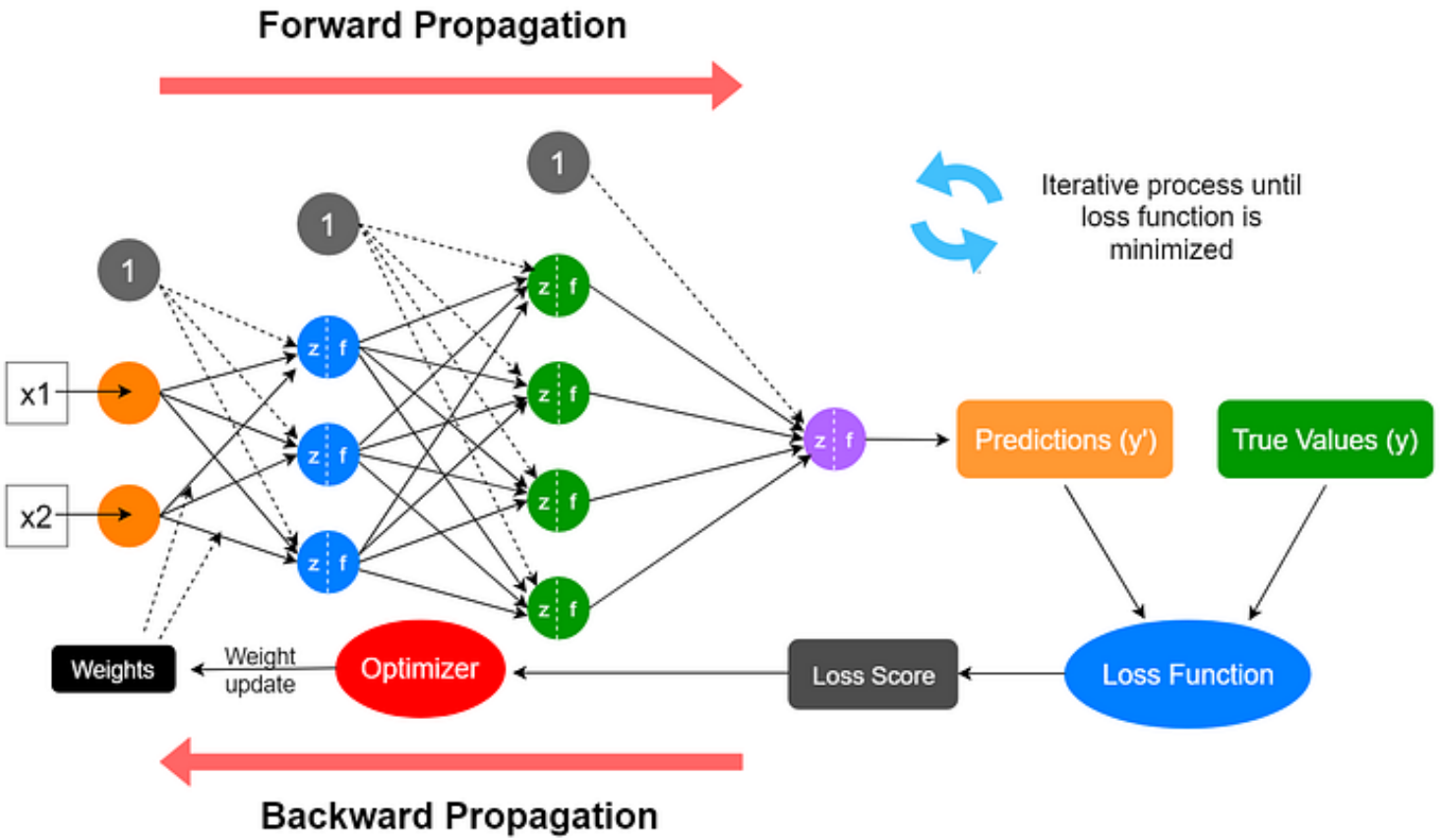
Perceptrons (Artificial Neurons) in Neural Networks





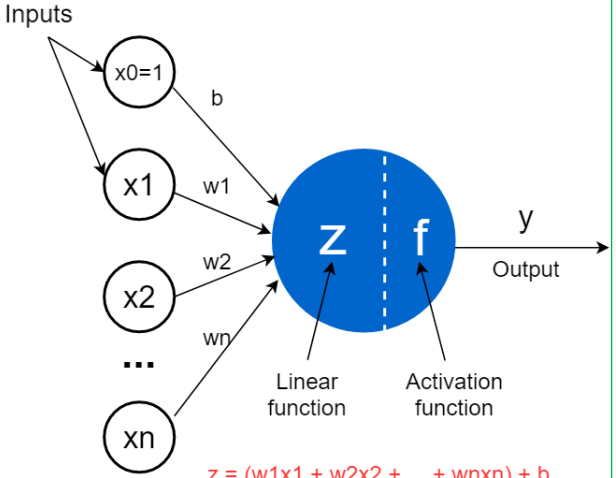
https://towardsdatascience.com/the-concept-of-artificial-neurons-perceptrons-in-neural-networks-fab22249cbfc/

Overview of a Neural Network's Learning Process



https://medium.com/data-science-365/overview-of-a-neural-networks-learning-process-61690a502fa

Activation Functions



z = (w1x1 + w2x2 + ... + wnxn) + b

y = f(z)

z = (w1.x1 + w2.x2 + … + wn.xn) + b

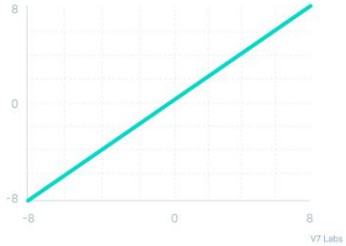
y = f(z)

y = f((w1.x1 + w2.x2 + ... + wn.xn) + b)

Output — Activation function (non-linear) — Weighted sum of inputs (linear combination) — Bias

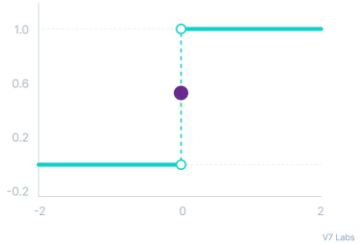
Linear

$$f(x) = x$$



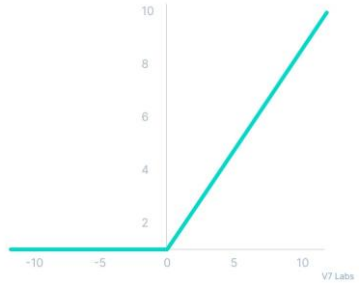
Binary step

$$f(x) = \begin{cases} 0 & for\ x < 0 \\ 1 & for\ x \geqslant 0 \end{cases}$$



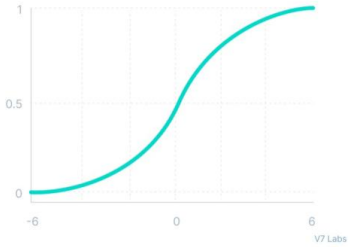
ReLu (rectifying nonlinearity)

$$f(x) = max\ (0, x)$$



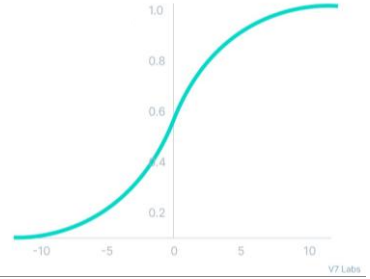
Sigmoid / Logistic

$$f(x) = \frac{1}{1+e^{-x}}$$



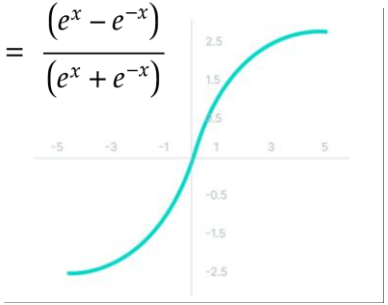
Softmax

$$f(x) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$



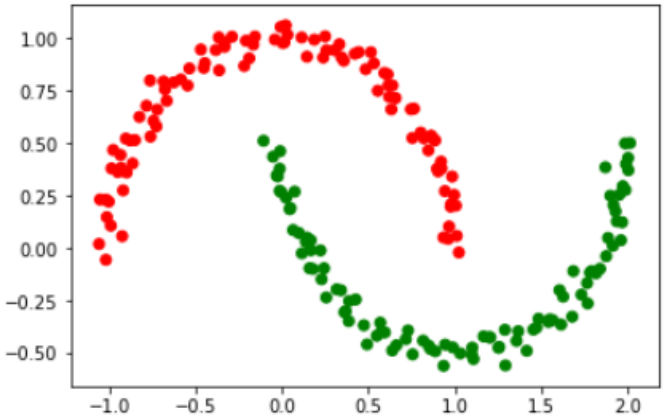
Tanh (tangens hyperbolicus)

$$f(x) = \frac{\left(e^{x} - e^{-x}\right)}{\left(e^{x} + e^{-x}\right)}$$

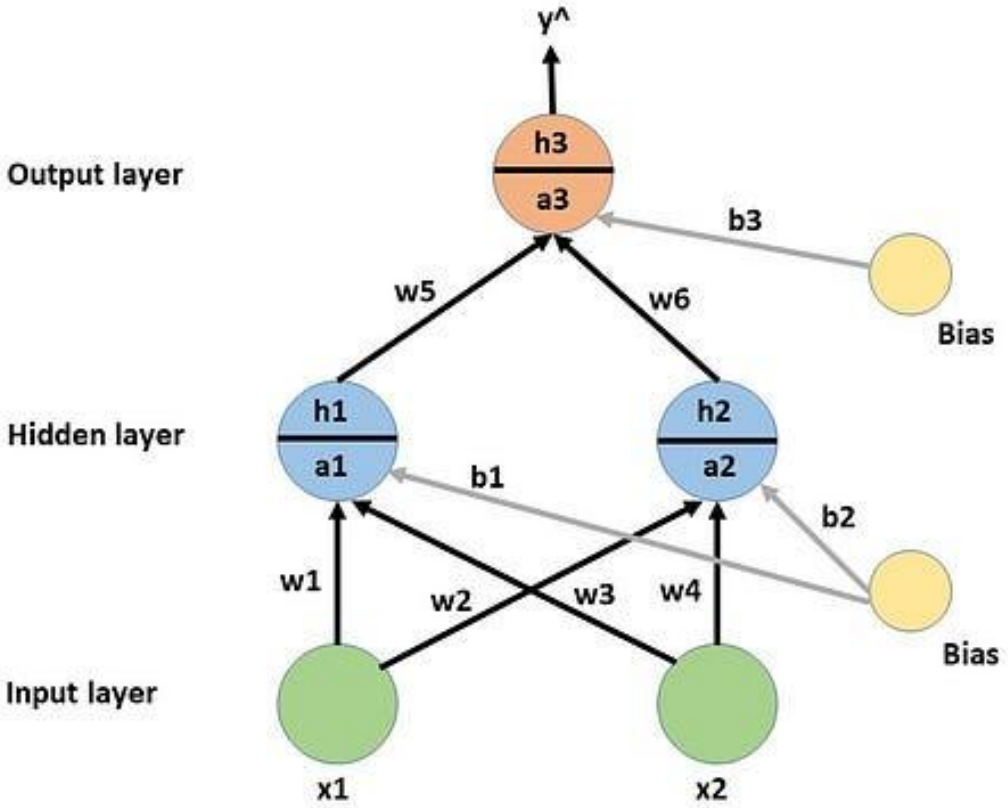


ตัวอย่างคำนวณ #1

200 samples are used to generate the data and it has two classes shown in red and green color.

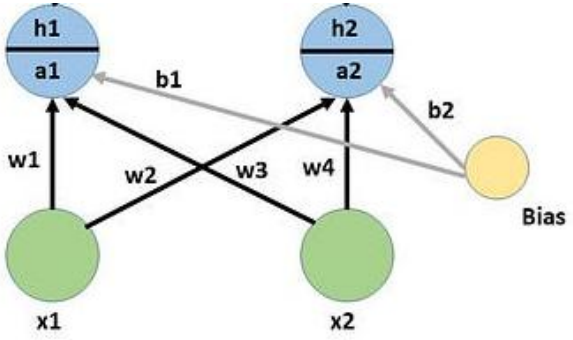


Using 1 hidden layer with 2 neurons, an output layer with a single neuron and sigmoid activation function.



ตัวอย่างคำนวณ #1

- During forward propagation at each node of hidden and output layer preactivation and activation takes place.
- For example, a1 is calculated first and then h1 is calculated.
- Initially, the weights are randomly generated.



x1 = 0.88 x2 = (-0.49)

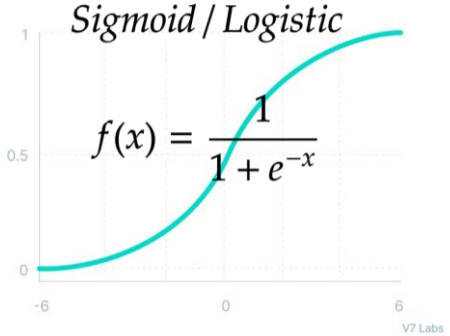
a1 is a weighted sum of inputs.

a1 = w1*x1 + w3*x2 + b1 = 1.76* 0.88 + 0.40*(-0.49) + 0 = 1.37

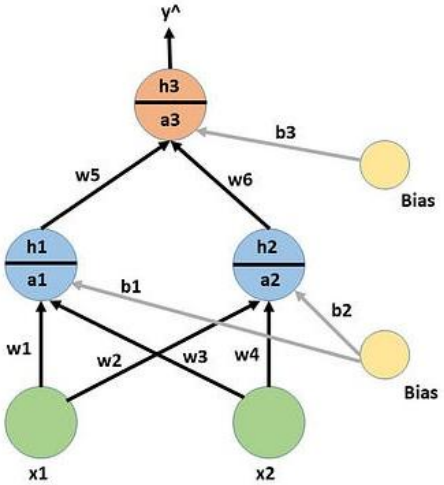
h1 = 1 / (1 + e^{-a1}) = 0.8

a2 = w2*x1 + w4*x2 + b2 = 0.97 *0.88 + 2.24 *(- 0.49)+ 0 = -2.29

h2 = 1 / (1 + e^{-a2}) = 0.44



ตัวอย่างคำนวณ #1

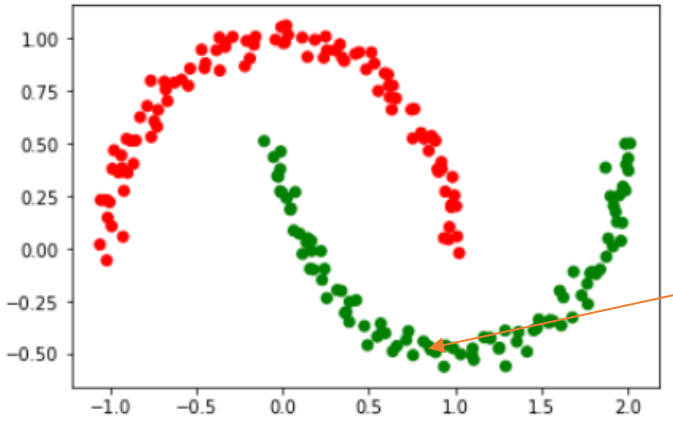


- For any layer after the first hidden layer, the input is output from the previous layer.

  $a3 = w5*h1 + w6*h2 + b3 = 1.86*0.8 + (-0.97)*0.44 + 0 = 1.1$

  $h3 = 1 / (1 + e^{-a3}) = 0.74$

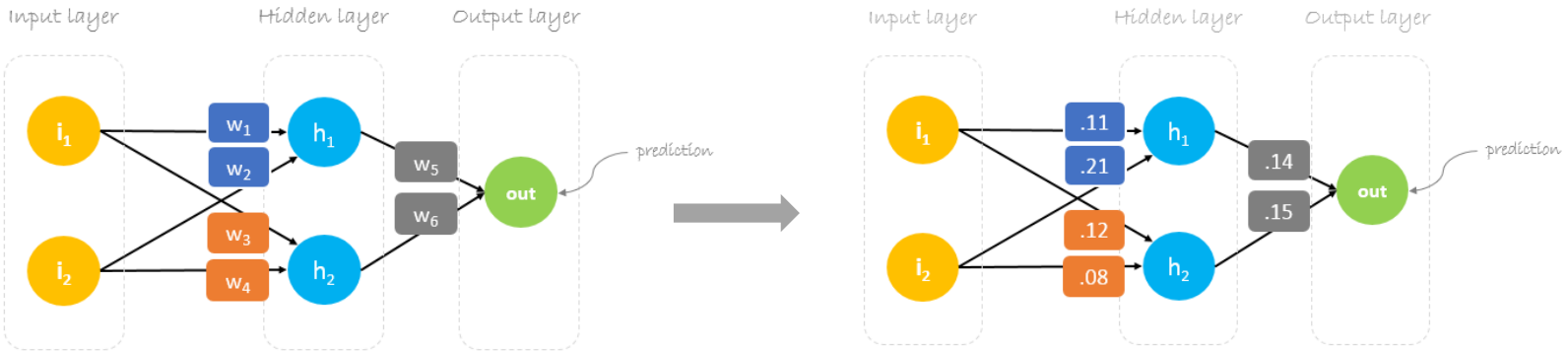
- So there are 74% chances the first observation will belong to class 1.
- Like this for all the other observations predicted output can be calculated.



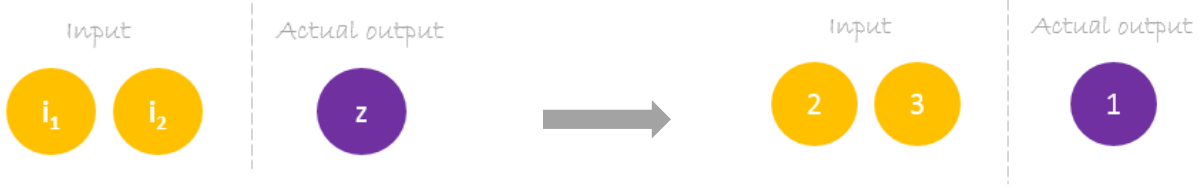
x1 = 0.88 x2 = (-0.49)

ตัวอย่างคำนวณ #2

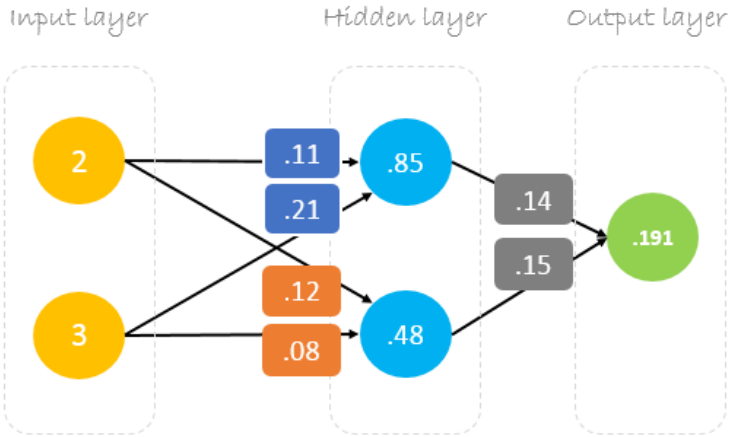
initial weights as following: w1 = 0.11, w2 = 0.21, w3 = 0.12, w4 = 0.08, w5 = 0.14 and w6 = 0.15



dataset has one sample with two inputs and one output. inputs=[2, 3] and output=[1]



Forward Pass



Forward Pass

Matrix multiplication

$$[2 \quad 3] \cdot \begin{bmatrix} 0.11 & 0.12 \\ 0.21 & 0.08 \end{bmatrix} = [0.85 \quad 0.48] \cdot \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} = [0.191]$$

Details

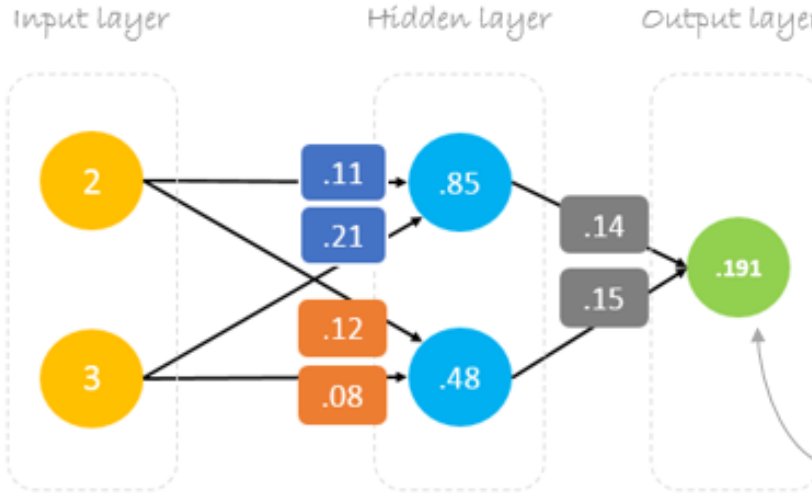
2 x .11 + 3 x .21 = .85

2 x .12 + 3 x .08 = .48

.85 x .14 + .48 x .15 = .191

Calculating Error

$$\text{MSE} = \frac{1}{2}(\hat{y} - y)^2$$



Input: 2, 3

Actual output: 1

prediction

actual

Error = 0 , if prediction = actual

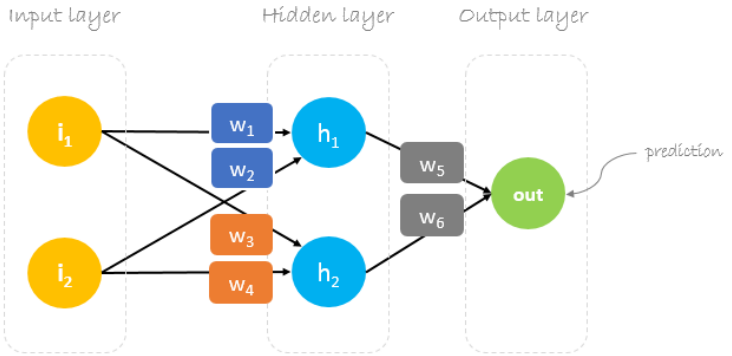
Error = $\frac{1}{2}$(prediction – actual)²

Error is always positive because of the square

$\frac{1}{2}$ is added to ease the calculation of the derivative

Error = $\frac{1}{2}$(0.191 – 1.0)² = 0.327

Reducing Error



$$\textbf{prediction} = \text{out}$$

$$\textbf{prediction} = (h_1)\, w_5 + (h_2)\, w_6$$

$$h_1 = i_1 w_1 + i_2 w_2$$

$$h_2 = i_1 w_3 + i_2 w_4$$

$$\textbf{prediction} = (i_1 w_1 + i_2 w_2)\, w_5 + (i_1 w_3 + i_2 w_4)\, w_6$$

to change ***prediction*** value,
we need to change ***weights***

Backpropagation

- short for “backward propagation of errors”
- a mechanism used to update the **weights** using gradient descent.
- It calculates the gradient of the error function with respect to the neural network’s weights. The calculation proceeds backwards through the network.

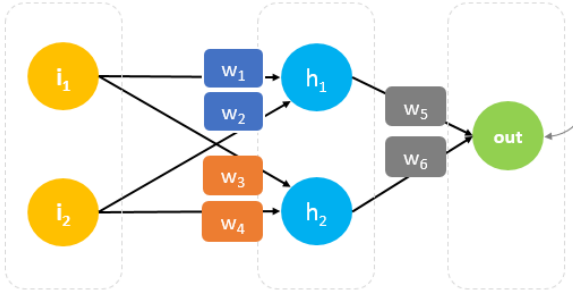
Old weight

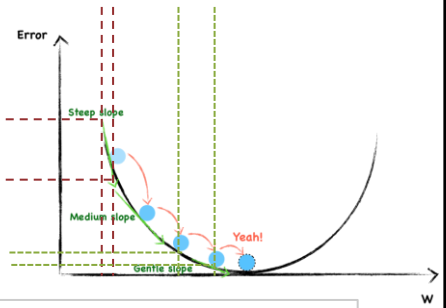
Derivative of Error with respect to weight

$$^{*}W_x = W_x - a\left(\frac{\partial Error}{\partial W_x}\right)$$

New weight

Learning rate





Gradient descent is an iterative optimization algorithm for finding the minimum of a function; in our case we want to minimize the error function. To find a local minimum of a function using gradient descent, one takes steps proportional to the negative of the gradient of the function at the current point.

Backpropagation

- To update w6, we take the current w6 and subtract the partial derivative of error function with respect to w6.

$$^{*}W_6 = W_6 - a\left(\frac{\partial Error}{\partial W_6}\right)$$

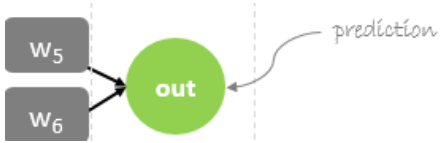
สมมติ $Error = w^2$

$$\frac{dError}{dw} = 2w$$

- Optionally, we multiply the derivative of the error function by a selected number to make sure that the new updated weight is minimizing the error function; this number is called learning rate.
- The derivation of the error function is evaluated by applying the chain rule as following

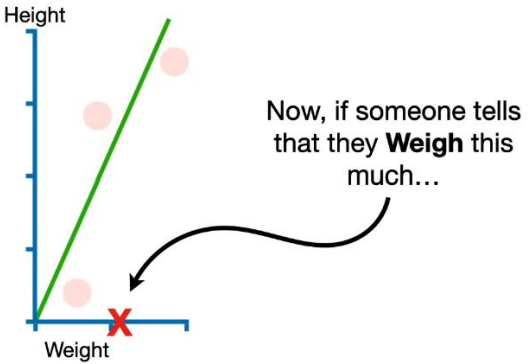
$$\frac{\partial Error}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

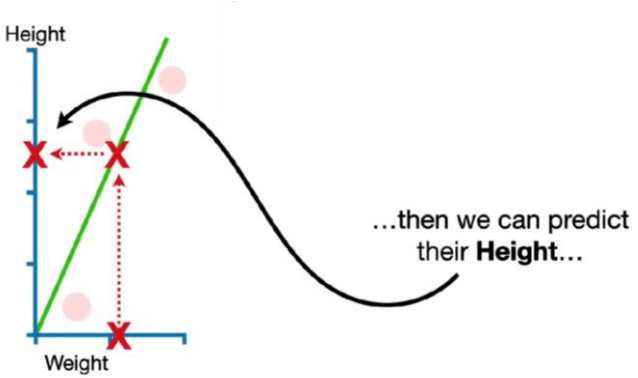
chain rule

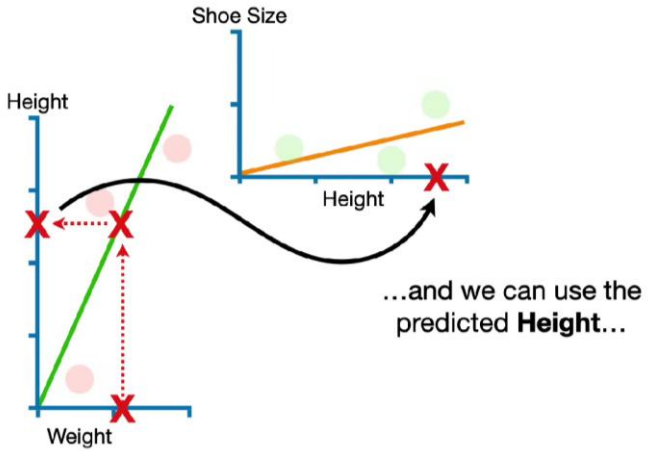


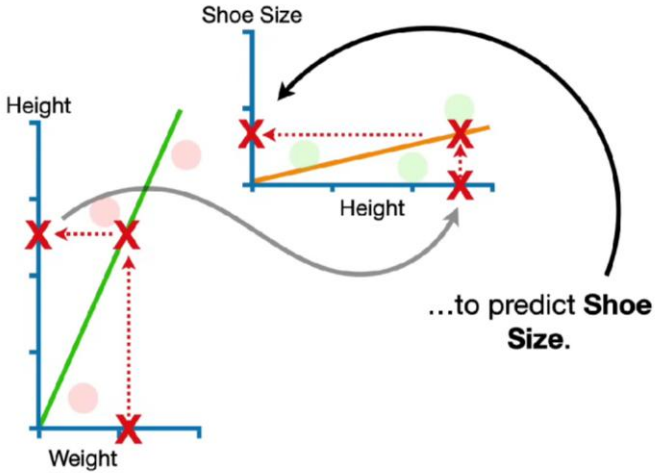
$$\text{MSE} = \frac{1}{2}(\hat{y} - y)^2$$

The Chain Rules









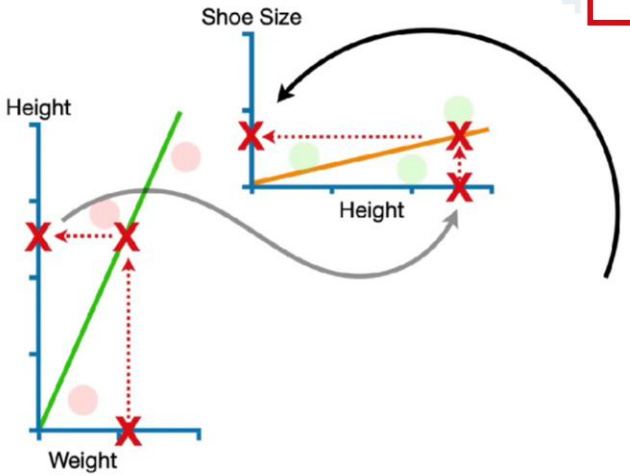
The Chain Rules

$$\text{Shoe Size} = \frac{dSize}{dHeight} \times \text{Height}$$

$$\text{Shoe Size} = \frac{dSize}{dHeight} \times \frac{dHeight}{dWeight} \times \text{Weight}$$

$$\text{Height} = \frac{dHeight}{dWeight} \times \text{Weight}$$

…we can plug the equation for **Height** into the equation for **Shoe Size**.



Backpropagation

$$\frac{\partial Error}{\partial W_6} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial W_6}$$

chain rule

$$Error = \frac{1}{2}(prediction - actual)^2$$

$$prediction = (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6$$

$$\frac{\partial Error}{\partial W_6} = \frac{\frac{1}{2}(\ prediction - actual\ )^2}{\partial prediction} * \frac{\partial\ (i_1 w_1 + i_2 w_2) w_5 + (i_1 w_3 + i_2 w_4) w_6}{\partial W_6}$$

$$\frac{\partial Error}{\partial W_6} = 2 * \frac{1}{2}(\ \textbf{prediction - actual}\ ) \frac{\partial(\ prediction - actual\ )}{\partial prediction} * (i_1 w_3 + i_2 w_4)$$

$h_2 = i_1 w_3 + i_2 w_4$

$$\frac{\partial Error}{\partial W_6} = (\ \textbf{prediction - actual}\ ) * (h_2)$$

Δ = prediction – actual ← delta

$$\frac{\partial Error}{\partial W_6} = \Delta h_2$$

So to update w6 we can apply the following formula :

$$^*W_6 = W_6 - a\left(\frac{\partial Error}{\partial W_6}\right)$$

$$^*W_6 = W_6 - a\,\Delta h_2$$

Backpropagation

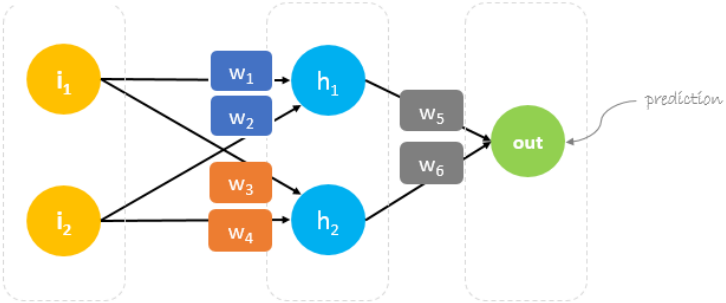
- Similarly, we can derive the update formula for w5

$$^{*}W_5 = W_5 - a\,\Delta h_1$$

However, when moving backward to update w1, w2, w3 and w4 existing between input and hidden layer, the partial derivative for the error function with respect to w1, for example, will be as following.

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$

chain rule



Backpropagation

$$\frac{\partial Error}{\partial W_1} = \frac{\partial Error}{\partial prediction} * \frac{\partial prediction}{\partial h_1} * \frac{\partial h_1}{\partial W_1}$$ ← chain rule

$$\frac{\partial Error}{\partial W_1} = \frac{\partial \frac{1}{2}(\ prediction - actual\ )^2}{\partial prediciton} * \frac{\partial\ (h_1)\ w_5 + (h_2)\ w_6}{\partial h_1} * \frac{\partial\ i_1\ w_1 + i_2\ w_2}{\partial w_1}$$

$$\frac{\partial Error}{\partial W_1} = 2 * \frac{1}{2}(\quad \textbf{prediction - actual} \quad) \frac{\partial(\ prediction - actual\ )}{\partial prediciton} * (w_5) * (i_1)$$

$$\frac{\partial Error}{\partial W_1} = (\quad \textbf{prediction - actual} \quad) * (w_5 i_1)$$

$$\frac{\partial Error}{\partial W_1} = \Delta\ w_5 i_1$$

Error = $\frac{1}{2}$(prediction – actual)^2

prediction = $(h_1)\ w_5 + (h_2)\ w_6$

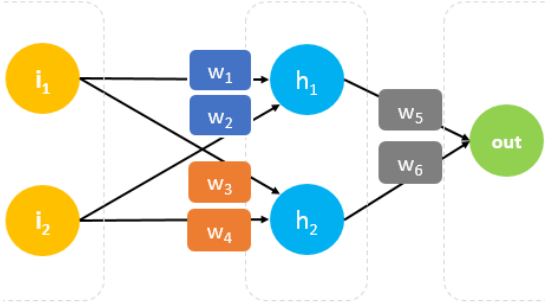
$h_1 = i_1 w_1 + i_2 w_2$

Δ = prediction – actual ← delta

Backpropagation

We can find the update formula for the remaining weights w2, w3 and w4 in the same way.

In summary, the update formulas for all weights will be as following:



$$^{*}w_6 = w_6 - a\,(h_2 \,.\, \Delta)$$

$$^{*}w_5 = w_5 - a\,(h_1 \,.\, \Delta)$$

$$^{*}w_4 = w_4 - a\,(i_2 \,.\, \Delta w_6)$$

$$^{*}w_3 = w_3 - a\,(i_1 \,.\, \Delta w_6)$$

$$^{*}w_2 = w_2 - a\,(i_2 \,.\, \Delta w_5)$$

$$^{*}w_1 = w_1 - a\,(i_1 \,.\, \Delta w_5)$$

$$\text{Error} = \frac{1}{2}(\text{prediction} - \text{actual})^2$$

$$\Delta = \text{prediction} - \text{actual}$$

We can rewrite the update formulas in matrices as following :

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - a\,\Delta \begin{bmatrix} h_1 \\ h_2 \end{bmatrix} = \begin{bmatrix} w_5 \\ w_6 \end{bmatrix} - \begin{bmatrix} a h_1 \Delta \\ a h_2 \Delta \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - a\,\Delta \begin{bmatrix} i_1 \\ i_2 \end{bmatrix} . \begin{bmatrix} w_5 & w_6 \end{bmatrix} = \begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} - \begin{bmatrix} a\, i_1 \Delta w_5 & a\, i_1 \Delta w_6 \\ a\, i_2 \Delta w_5 & a\, i_2 \Delta w_6 \end{bmatrix}$$

Backward Pass

- Using derived formulas we can find the new weights.
- Learning rate is a hyperparameter which means that we need to manually guess its value.

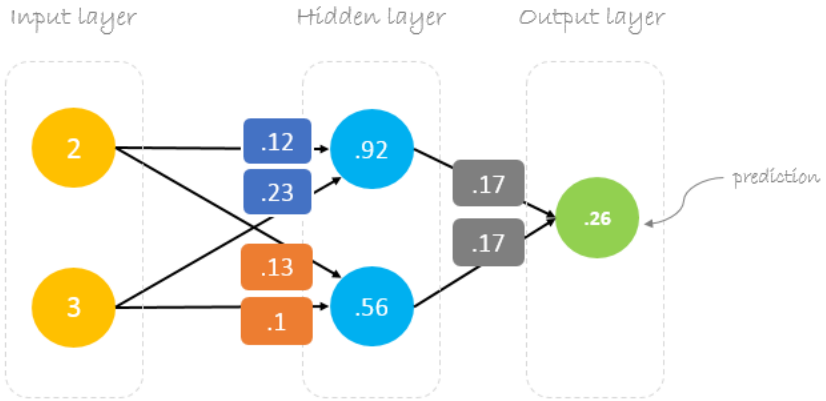
$\Delta = 0.191 - 1 = -0.809$ ← **Delta** = prediction - actual

$a = 0.05$ ← **Learning rate,** we smartly guess this number

$$\begin{bmatrix} w_5 \\ w_6 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 0.85 \\ 0.48 \end{bmatrix} = \begin{bmatrix} 0.14 \\ 0.15 \end{bmatrix} - \begin{bmatrix} -0.034 \\ -0.019 \end{bmatrix} = \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix}$$

$$\begin{bmatrix} w_1 & w_3 \\ w_2 & w_4 \end{bmatrix} = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - 0.05(-0.809) \begin{bmatrix} 2 \\ 3 \end{bmatrix} . [0.14 \quad 0.15] = \begin{bmatrix} .11 & .12 \\ .21 & .08 \end{bmatrix} - \begin{bmatrix} -0.011 & -0.012 \\ -0.017 & -0.018 \end{bmatrix} = \begin{bmatrix} .12 & .13 \\ .23 & .10 \end{bmatrix}$$

Forward pass with new weights



Forward Pass

We can notice that the **prediction** 0.26 is a little bit closer to **actual output** than the previously predicted one 0.191.
We can repeat the same process of backward and forward pass until **error** is close or equal to zero.

Matrix multiplication

$$[2 \quad 3] \cdot \begin{bmatrix} 0.12 & 0.13 \\ 0.23 & 0.10 \end{bmatrix} = [0.92 \quad 0.56] \cdot \begin{bmatrix} 0.17 \\ 0.17 \end{bmatrix} = [0.26]$$

Details

2 x .12 + 3 x .23 = .85

2 x .13 + 3 x .10 = .48

.92 x .17 + .56 x .17 = .26

Neural Networks – Updating Weights

Cost function / Lost function

- Any function that measures an error can be used as a cost function.

Regression Loss Functions:

- Mean Absolute Error (MAE aka. L1 Loss)
- Huber Loss (combines L1 and L2)
- Log-Cosh Loss
- Quantile Loss
- Poisson Loss
- .....

Classification Loss Functions:

- Cross-Entropy Loss (also known as Log Loss)
- Binary Cross-Entropy Loss
- Categorical Cross-Entropy Loss Hinge Loss
- KL Divergence
- Hinge Loss
- ....

Loss Function: MSE

สำหรับ sample เดียว:

$$\text{MSE} = \frac{1}{2}(\hat{y} - y)^2$$

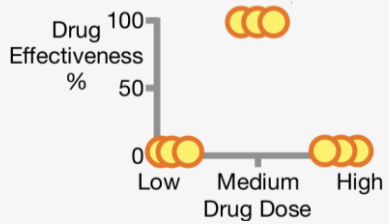
ถ้า batch size = m, loss รวมคือ:

$$\text{MSE}_{\text{batch}} = \frac{1}{2m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2$$

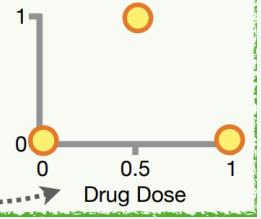
ตัวอย่างคำนวณ #3

Activation Function = Softplus

Loss Function = Sum of the Square Residuals



NOTE: To keep the math relatively simple, from now on, the **Training Data** will have only **3 Dose** values, **0**, **0.5**, and **1**.



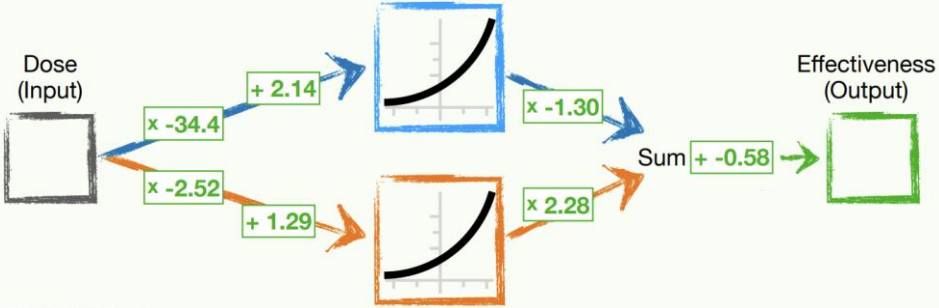
Drug Dose	Predicted	Actual
0	0.57	0
0.5	1.61	1
1	0.58	0

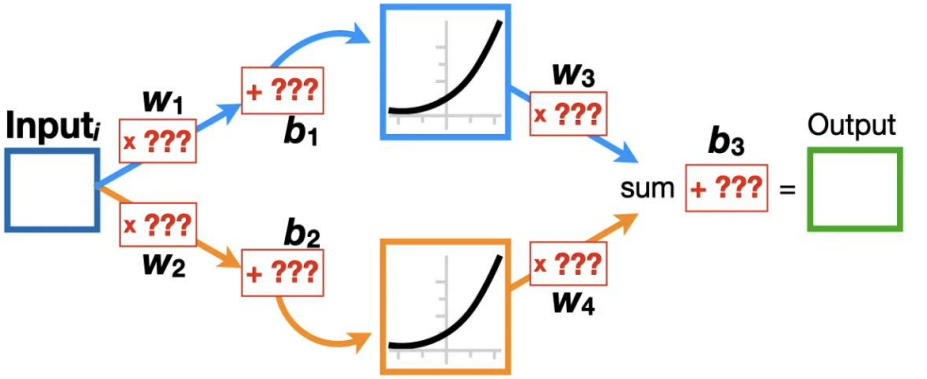
SSR = (0-0.57)^2 + (1-1.61)^2 + (0-0.58)^2 = 1.0

Softplus เป็นเวอร์ชัน smooth ของ ReLU

Softplus

$$Softplus(x) = \ln(1 + e^x)$$





Backpropagation: Updating b3

$$(Observed_i - Predicted_i)^2$$

$$ed_i$$

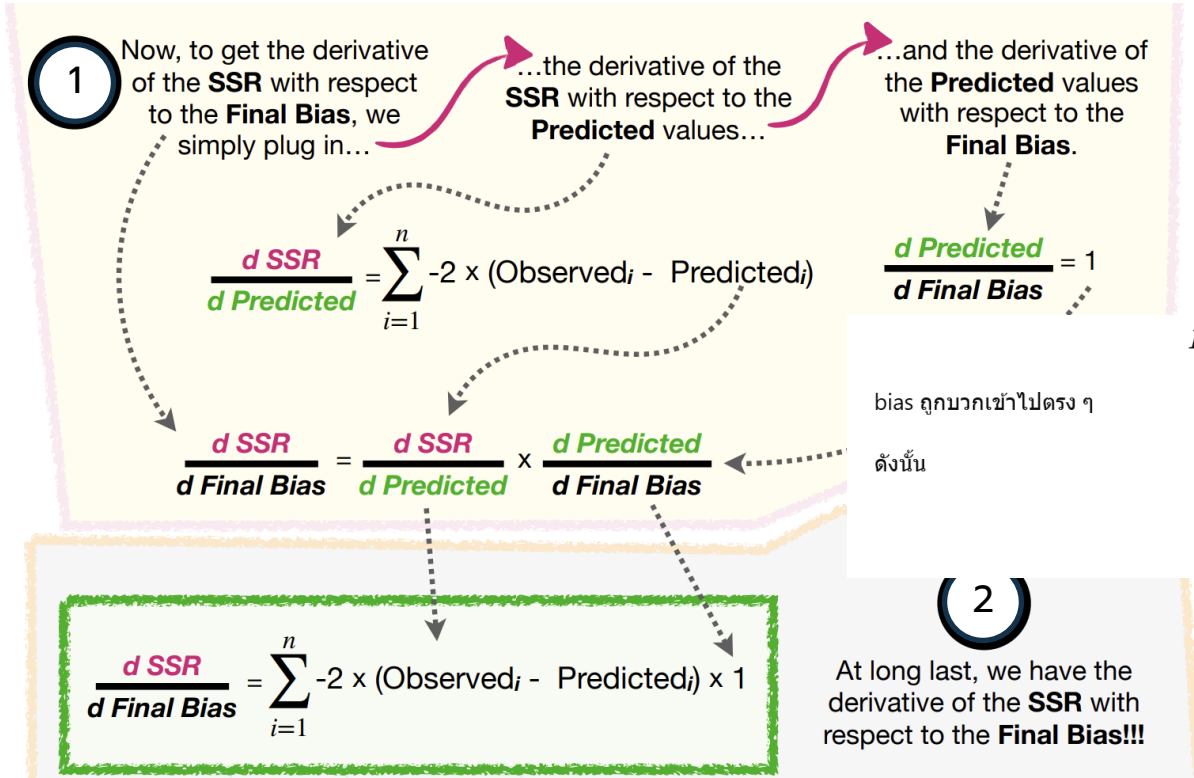
$$\frac{\partial}{\partial Predicted_i}(Observed_i - Predicted_i)^2$$

$$- Predicted_i) \cdot \frac{\partial}{\partial Predicted_i}(Observed_i - Predicted_i)$$

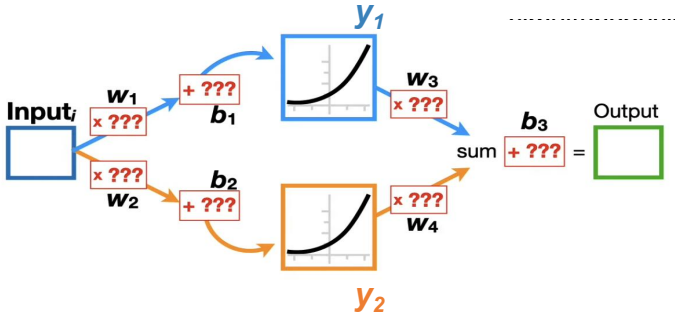
$$\frac{\partial}{redicted_i}(Observed_i - Predicted_i) = -1$$

$$erved_i - Predicted_i)^2 = -2(Observed_i - Predicted_i)$$

$$\frac{SSR}{dicted} = \sum_{i=1}^{n} -2(Observed_i - Predicted_i)$$



Backpropagation: Updating w3, w4



$$\frac{d\,SSR}{d\,w_3} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,w_3}$$

$$\frac{d\,Predicted}{d\,w_3} = \frac{d}{d\,w_3}(y_{1,i}w_3 + y_{2,i}w_4 + b_3) = y_{1,i}$$

$$\frac{d\,SSR}{d\,Predicted} = \frac{d}{d\,Predicted}\sum_{i}^{n=3}(Observed_1 - Predicted_i)^2 = \sum_{i=1}^{n=3} -2 \times (Observed_1 - Predicted_i)$$

$$\frac{d\,SSR}{d\,w_3} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,w_3} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times y_{1,i}$$

$$\frac{d\,SSR}{d\,w_4} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,w_4} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times y_{2,i}$$

Backpropagation: Updating b3, w3, w4

$$\frac{d\,SSR}{d\,w_3} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,w_3} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times y_{1,i}$$

$$\frac{d\,SSR}{d\,w_4} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,w_4} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times y_{2,i}$$

$$\frac{d\,SSR}{d\,b_3} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,b_3} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times 1$$

$\frac{d\,SSR}{d\,w_3} = 2.58$

$\frac{d\,SSR}{d\,w_4} = 1.26$

$\frac{d\,SSR}{d\,b_3} = 1.90$

Step Size = 2.58 x 0.1 = 0.258

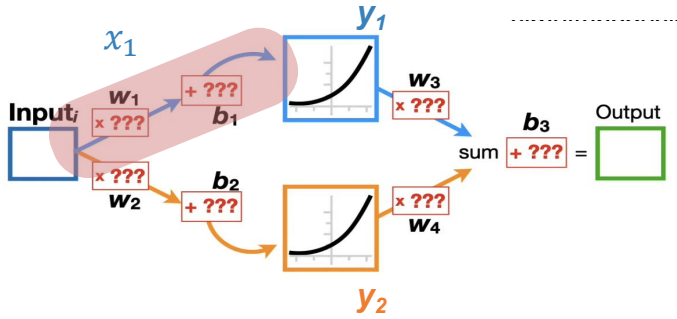
New w_3 = Old w_3 - Step Size

New w_3 = Old w_3 - Step Size

NOTE: In this example we've set the **Learning Rate** to **0.1**.

Backpropagation: Updating w1

$$\frac{d\,SSR}{d\,w_1} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,y_1} \times \frac{d\,y_1}{d\,x_1} \times \frac{d\,x_1}{d\,w_1}$$



$$Predicted_i = y_{1,i}w_3 + y_{2,i}w_4 + b_3 \rightarrow \frac{d\,Predicted}{d\,y_1} = w_3$$

$$X_{1,i} = Input_i \times w_1 + b_1 \rightarrow \frac{dx_1}{dw_1} = Input_i$$

Activation Function = Softplus

$$y_{1,i} = f(x_{1,i}) = \log(1 + e^x)_i$$

$$\frac{dy_1}{dx_1} = \frac{1}{1+e^x}\ x\ (0 + e^x) = \frac{e^x}{1+e^x}$$

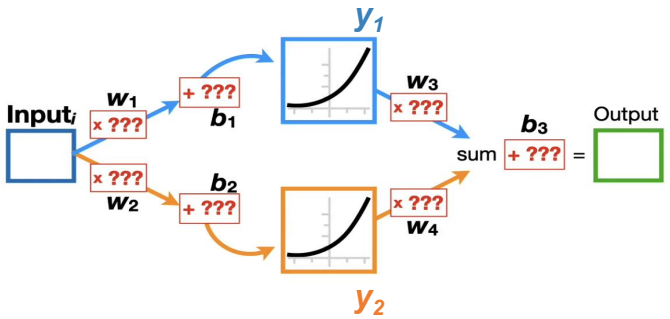
$$\frac{d\,SSR}{d\,w_1} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times w_3 \times \frac{e^x}{1 + e^x} \times Input_i$$

Backpropagation: Updating b1

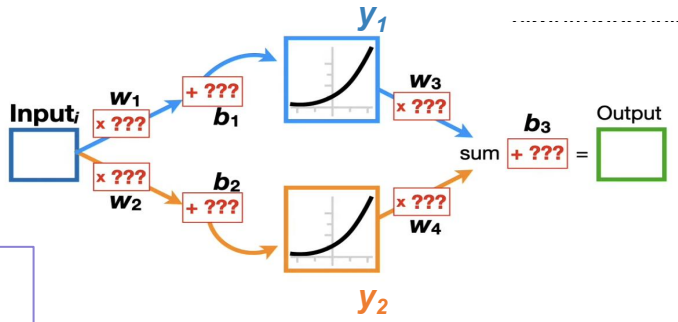
$$X_{1,i} = Input_i \times w_1 + b_1 \Rightarrow \frac{dx_1}{db_1} = 1$$

$$\frac{d\,SSR}{d\,b_1} = \frac{d\,SSR}{d\,Predicted} \times \frac{d\,Predicted}{d\,y_1} \times \frac{d\,y_1}{d\,x_1} \times \frac{d\,x_1}{d\,b_1}$$

$$\frac{d\,SSR}{d\,b_1} = \sum_{i=1}^{n=3} -2 \times (Observed_i - Predicted_i) \times w_3 \times \frac{e^x}{1+e^x} \times 1$$



Backpropagation: Updating w1, b1, w2, b2



$$\frac{d\,SSR}{d\,w_1} = \sum_{i=1}^{n=3} -2 \times (\text{Observed}_i - \text{Predicted}_i) \times w_3 \times \frac{e^x}{1+e^x} \times \text{Input}_i$$

$$\frac{d\,SSR}{d\,b_1} = \sum_{i=1}^{n=3} -2 \times (\text{Observed}_i - \text{Predicted}_i) \times w_3 \times \frac{e^x}{1+e^x} \times 1$$

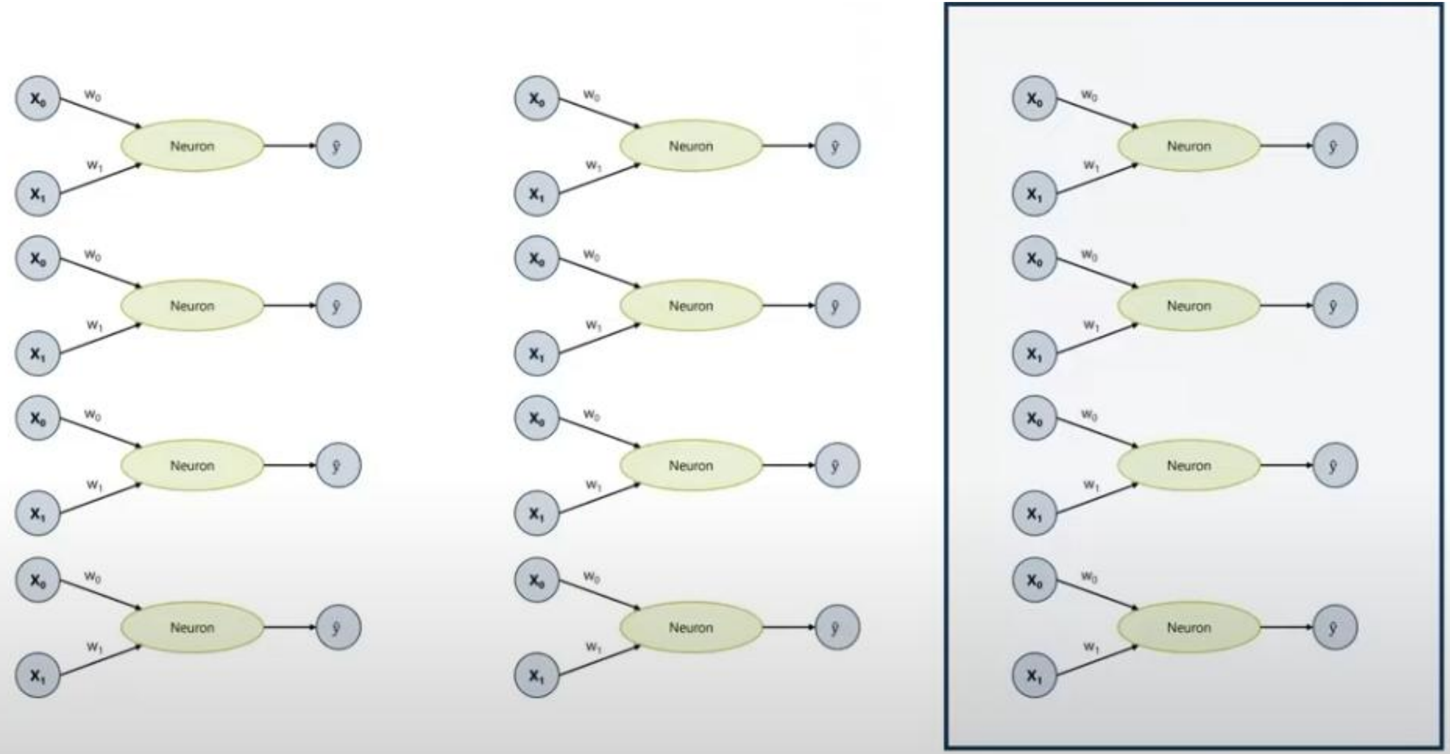
$$\frac{d\,SSR}{d\,w_2} = \sum_{i=1}^{n=3} -2 \times (\text{Observed}_i - \text{Predicted}_i) \times w_4 \times \frac{e^x}{1+e^x} \times \text{Input}_i$$

$$\frac{d\,SSR}{d\,b_2} = \sum_{i=1}^{n=3} -2 \times (\text{Observed}_i - \text{Predicted}_i) \times w_4 \times \frac{e^x}{1+e^x} \times 1$$

Neural Networks – Updating Weights - optimizer

- The problem with Gradient Descent:
  - We need to go through our whole dataset before we can adjust the weights --> takes a lot of time
- Stochastic Gradient Descent (SGD)
  - SGD only used a subset of the data to iteratively optimize the weights and biases.
  - Divide training data into batches.
  - Use only a part of training data (batch 1) to do a first adjustment of the weights
  - Feed the next batch of training data into the model and adjust the weights again right after the batch

Neural Networks – Updating Weights - optimizer



$$C^{approx} = \sum(\hat{y} - y)^2 \qquad C^{approx} = \sum(\hat{y} - y)^2 \qquad C^{approx} = \sum(\hat{y} - y)^2$$

Neural Networks

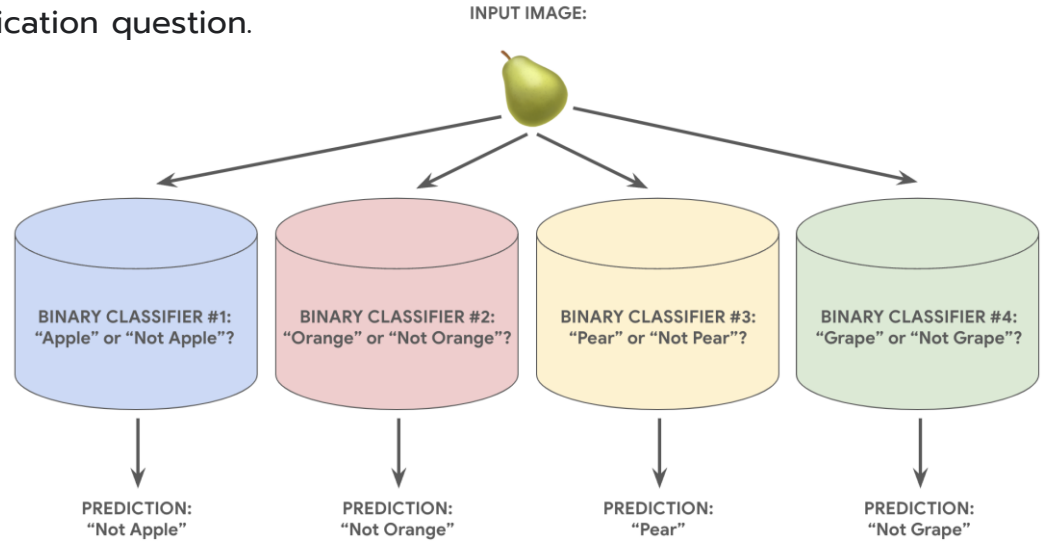
Definitions

- Epoch: One cycle through the full training data set
- Batch: A subset of the training data
- Loss function: Measures the difference between predicted output and the real value
- Learning Rate: Proportion of the gradient to calculate the weight's adjustment
- Optimizer: Method to adjust the model's parameters to minimize the loss function
- Activation function: A function used to introduce non-linearity into the network's calculations

Multiclass Classification

One versus all

- One-vs.-all provides a way to use binary classification for a series of yes or no predictions across multiple possible labels.
- Given a classification problem with N possible solutions, a one-vs.-all solution consists of N separate binary classifiers—one binary classifier for each possible outcome.
- During training, the model runs through a sequence of binary classifiers, training each to answer a separate classification question.

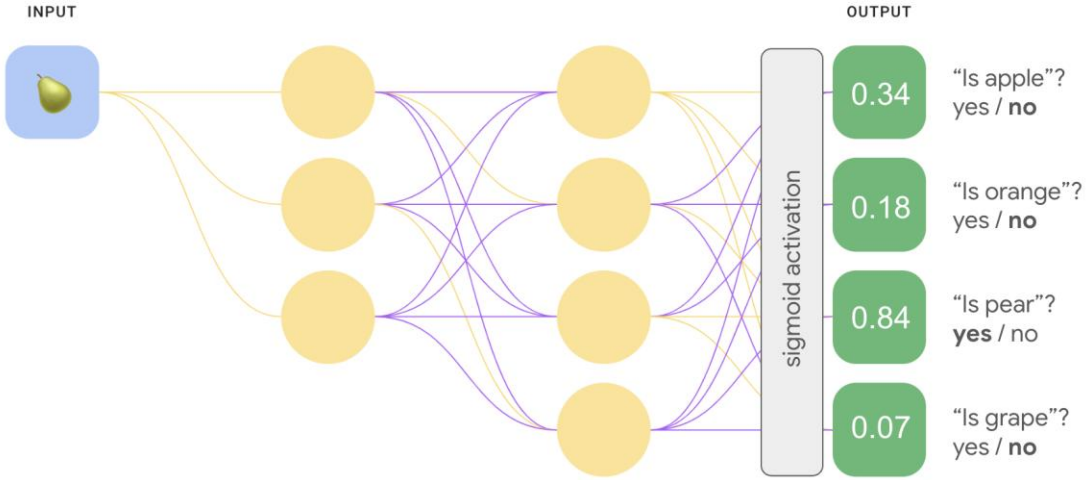


https://developers.google.com/machine-learning/crash-course/neural-networks/multi-class

Multiclass Classification

One versus all

- This approach is fairly reasonable when the total number of classes is small, but becomes increasingly inefficient as the number of classes rises.
- The same one-vs.-all classification tasks accomplished using a neural net model. A sigmoid activation function is applied to the output layer, and each output value represents the probability that the input image is a specified fruit. This model predicts that there is a 84% chance that the image is a pear, and a 7% chance that the image is a grape.

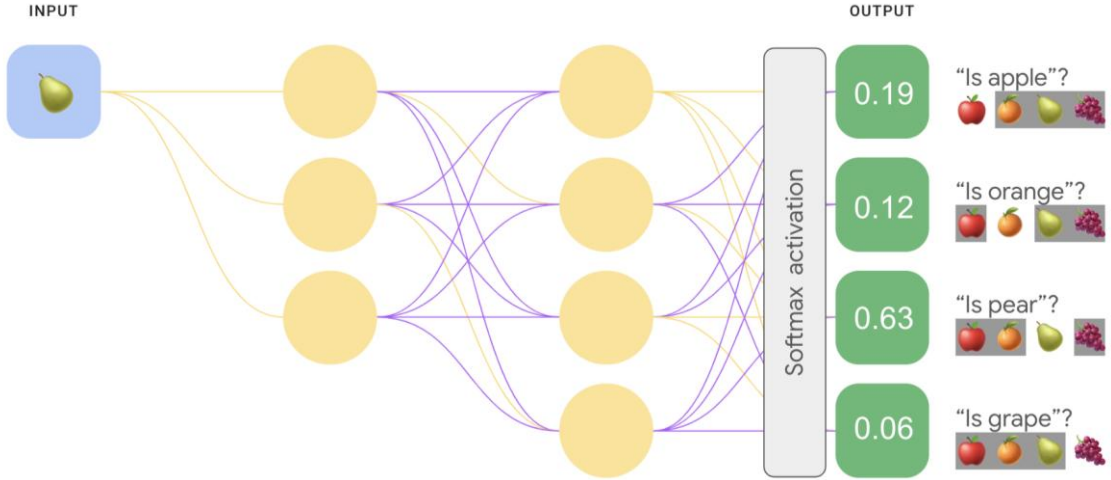


https://developers.google.com/machine-learning/crash-course/neural-networks/multi-class

Multiclass Classification

Using softmax

- Neural networks can perform multiclass classification directly by applying a softmax function at the output layer.
- Softmax converts the network's output scores into probabilities for each class.
- The probabilities of all classes sum to 1, forming a probability distribution.
- The class with the highest probability is selected as the prediction.



- Note that in order to perform softmax, the hidden layer directly preceding the output layer (called the softmax layer) must have the same number of nodes as the output layer.

sklearn.neural_network.MLPClassifier

```
class sklearn.neural_network.MLPClassifier(hidden_layer_sizes=(100,),
activation='relu', *, solver='adam', alpha=0.0001, batch_size='auto',
learning_rate='constant', learning_rate_init=0.001, power_t=0.5, max_iter=200,
shuffle=True, random_state=None, tol=0.0001, verbose=False, warm_start=False,
momentum=0.9, nesterovs_momentum=True, early_stopping=False, validation_fraction=0.1,
beta_1=0.9, beta_2=0.999, epsilon=1e-08, n_iter_no_change=10, max_fun=15000)
```
[source]

Multi-layer Perceptron classifier.

This model optimizes the log-loss function using LBFGS or stochastic gradient descent.

- activation = ใน MLPClassifier → ใช้กับ hidden layer
- Output layer sigmoid (Binary classification) / ใช้ softmax (Multiclass classification) โดยอัตโนมัติ

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Training Neural Networks

- After the neural network produces predictions, we need to measure how good the prediction is.
- This is done using a Loss Function.
- Loss Function = difference between predicted probability and true label
- For classification problems, the most commonly used loss function is Cross-Entropy.

sklearn.neural_network.MLPClassifier

Types of Cross-Entropy Loss Function

1. Binary Cross Entropy Loss

$$BCE = -\frac{1}{N}\Sigma_{i=1}^{N}(y_i.log(p_i) + (1 - y_i)log(1 - p_i))$$

where

- N is number of samples,
- y_i true label for sample i(0 or 1),
- p_i model-predicted probability for class 1 for sample i.

Used when the output layer uses sigmoid activation.

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

sklearn.neural_network.MLPClassifier

Types of Cross-Entropy Loss Function

2. Multiclass Cross Entropy Loss

- Multiclass Cross-Entropy Loss, also known as categorical cross-entropy or softmax loss.

$$CE = -\frac{1}{N}\Sigma_{i=1}^{N}\Sigma_{j=1}^{C}(y_{i,j}.log(p_{i,j}))$$

where

- N is number of samples,
- C is the number of classes.
- y_{ij} is 1 if class j is correct for sample i, 0 otherwise.
- p_{ij} is model-predicted probability of sample i being in class j.

Used when the output layer uses softmax activation.

https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

TensorFlow

Neural networks can be implemented using machine learning frameworks such as TensorFlow.

- TensorFlow is an open-source machine learning framework developed by Google
- Keras API: TensorFlow's high-level API, Keras, simplifies the process of building and training neural networks. It provides a user-friendly interface to define model architectures, configure training parameters, and manage data.
- Model Building: Neural networks in TensorFlow are typically constructed by stacking layers using the `tf.keras.Sequential` or `tf.keras.Model` API.
- Common layers include:
  - Dense (Fully Connected) Layers: Where each neuron in a layer is connected to every neuron in the preceding layer.
  - Convolutional Layers: Used in Convolutional Neural Networks (CNNs) for processing image data.
  - Recurrent Layers: Used in Recurrent Neural Networks (RNNs) for sequential data like text or time series.

TensorFlow

- Compilation: After defining the model architecture, it needs to be compiled by specifying:
  - Optimizer: An algorithm (e.g., Adam, SGD) that adjusts the model's weights during training to minimize the loss function.
  - Loss Function: A metric that quantifies the difference between the model's predictions and the actual target values (e.g., categorical_crossentropy for classification, mean_squared_error for regression).
  - Metrics: Used to monitor the training process and evaluate model performance (e.g., accuracy).